Case study by Chandana Vishnu Jayanth · vishnuatgit.github.io · jayanthvishnu9@gmail.com
Live demo: supplytrace-fkpq.onrender.com · Code: github.com/vishnuatgit/SupplyTrace

SupplyTrace

A modular pipeline that ingests raw XML invoices, validates them with deterministic rules, flags duplicates and routes doubtful ones to a review queue, with an experimental ML risk score and a vendor directory.

YEAR

2026

BUILT WITH

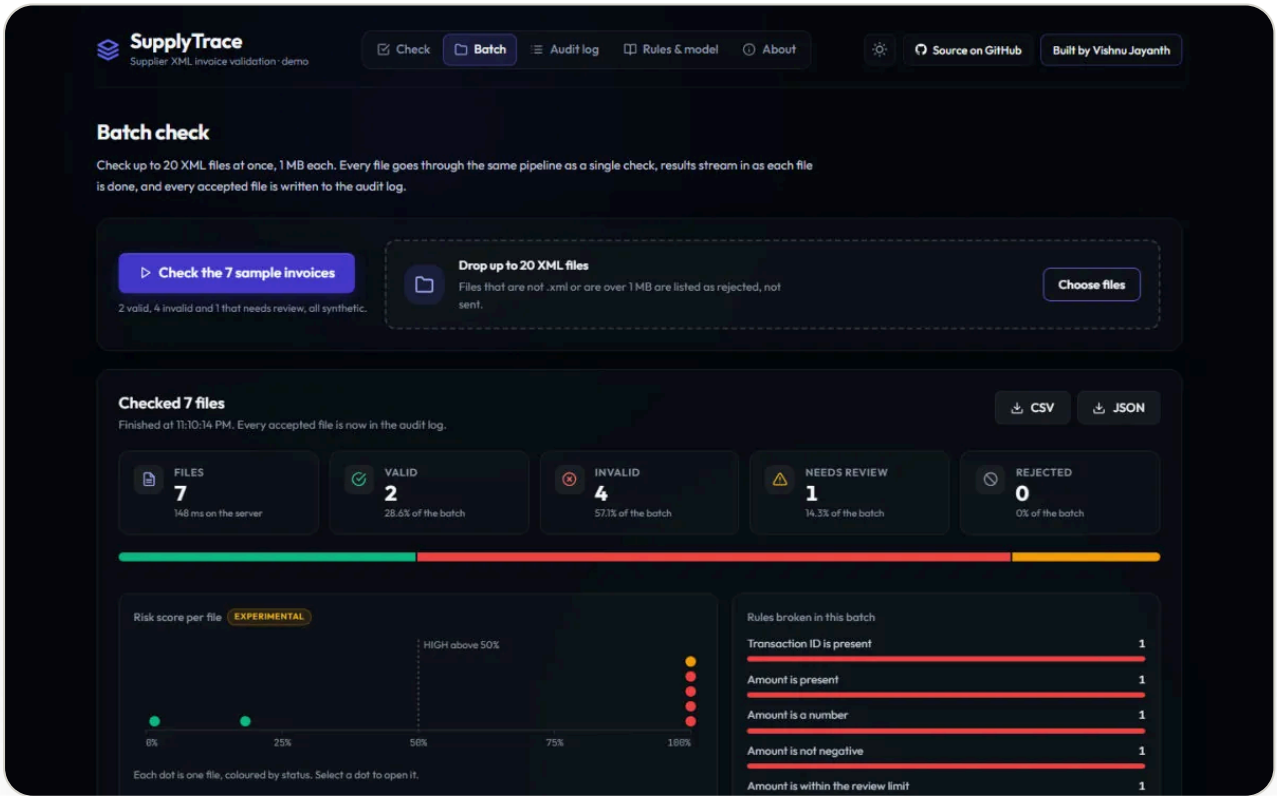
FastAPI · scikit-learn · Pandas

PROOF

- 233 tests · CI passing

SOURCE

github.com/vishnuatgit/SupplyTrace



(01) The problem

WHY IT EXISTS

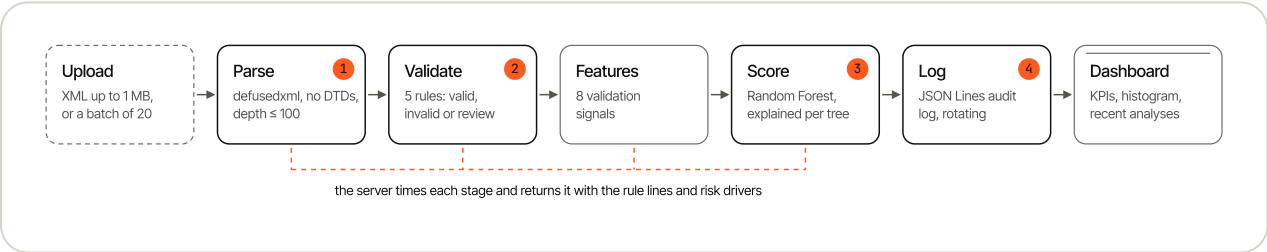
A bad invoice usually causes trouble *later,* in a system downstream.

By then it is hard to trace back to the file. SupplyTrace checks each file when it arrives: it labels every invoice valid, invalid or needs review with deterministic rules, points each rule at the line of the file behind it, and logs the result.

An experimental Random Forest adds a risk score trained on 400 synthetic invoices, with an explanation of what drove each prediction.

(02) How it works

FROM THE README



1 **Parse safely.** The upload is parsed in memory with `defusedxml` (no DTDs), files nested more than 100 levels deep are rejected, and uploads over 1 MB get a `413`. The XML is flattened into dot-notation keys, and a second `expat` pass records each element's line.

2 **Validate with rules.** Five deterministic rules decide the status: `VALID`, `INVALID` or `REQUIRES_REVIEW`. Every rule comes back as a checklist row with the line that shows it.

3 **Score the risk (experimental).** Eight validation signals go to a Random Forest trained on the synthetic batch; the score comes with the base rate plus one contribution per feature.

4 **Catch duplicates.** Each invoice's SHA-256 fingerprint and key fields are stored. The same content is an exact duplicate; the same supplier and invoice ID, or the same supplier and amount within 7 days, is a likely one and goes to review.

5 **Log it.** One JSON line per file goes to a rotating audit log, which also feeds the dashboard.

6 **Review.** `REQUIRES_REVIEW` files and likely duplicates land in a review queue: assign a reviewer, comment, and approve or reject with a required comment, with an append-only history. A vendor directory rolls the invoices up per supplier.

(03) Key decisions

6 CHOICES

01 Rules set the status; the model adds a risk score

The status always comes from five deterministic rules. The risk score is labelled experimental: it is trained on synthetic data and says nothing about real suppliers.

02 Safe parsing of untrusted uploads

`defusedxml` refuses DTDs, so there is no entity expansion or external entities; the 1 MB cap is enforced before the body is read; documents nested more than 100 levels deep are rejected.

03 Explaining each risk score

The explanation follows each tree from its root to the file's leaf and credits every split's change in failure rate to the feature it tested (tree-path attribution). The base rate plus the contributions equals the score, and the tests check it.

04 One code path and an audit log

A single check, a batch and a streamed batch all call the same `analyze_document()`, so a file gets the same answer every way (tests compare them). Every result is one line in a rotating JSON Lines audit log.

05 Compared with a rules-only baseline

The server trains a fresh model on 300 invoices and tests it on the other 100 next to a baseline that fails a file exactly when the rules do. At the 50% threshold the model makes the same call as the rules on every held-out file.

06 An append-only review trail

Reviews live in SQLite through SQLAlchemy Core, and any `DATABASE_URL` works. Every action adds a history row, a second decision gets a `409`, and “Send to review” re-analyses the file on the server, so a client cannot post a made-up result. A vendor’s trusted flag is shown but never changes a validation result, and a test checks that.

(04) Proof

TESTS · CI · STACK

233	Passing	400
PYTEST TESTS, NO NETWORK	CI ON PYTHON 3.11 AND 3.12, WITH RUFF	SYNTHETIC INVOICES THE RISK MODEL TRAINS ON

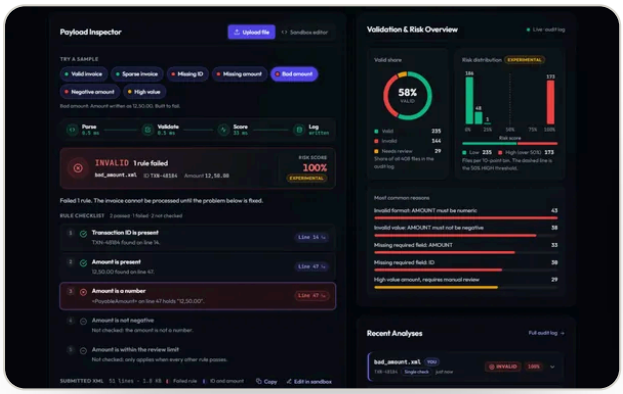
The suite covers XML parsing and safety (DTDs, entity expansion, depth and size limits, parse-error positions), every validation rule and every example on the rules page, the risk explanation (it must add up to the score), the single, batch and streamed endpoints (including that they agree), the audit log and the model card's

holdout metrics. Review state changes, duplicate detection on crafted invoices and the vendor statistics are tested end to end through the API. CI runs ↗

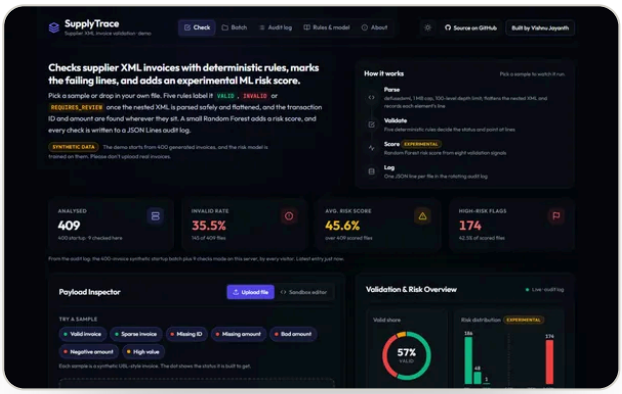
`Python 3.11+` `FastAPI` `Uvicorn` `defusedxml` `SQLAlchemy` `SQLite` `pandas` `NumPy` `scikit-learn` `pytest` `pytest-cov` `Ruff` `GitHub Actions`

(05) Screenshots

FROM THE REPOSITORY



A check: the verdict, the rule that failed and the line behind it.



The home page: the dashboard numbers come from the audit log.

(06) Limitations

HONESTLY

— **Synthetic training data.** The risk model is trained on synthetic invoices with simulated outcomes; real use would need outcome data, and the scores are not calibrated probabilities.

— **Five rules only:** no checks on line totals, tax, dates or currencies. Duplicate matching compares suppliers as written, so “Acme Corp” and “Acme Corporation” differ.

— **Flattening loses detail.** Repeated elements, such as several invoice lines, collapse to the last one, and XML attributes are ignored.

— **One shared audit log and review database**, reset on every restart on the free host unless `DATABASE_URL` points at a hosted Postgres.

— **No saved model.** It is retrained at every startup, and there are no accounts or authentication, so reviewer names are self-declared.

NEXT CASE STUDY

ArchLens ↗

Scores any public GitHub repository from 0 to 100 across seven engineering dimensions, as a web app, a REST API and a CLI.